

任务 3.1: 编写自定义业务状态码与统一响应结构

在现代前后端分离开发中，无论业务成功与否，HTTP 状态码通常保持 200，真实的业务结果需要通过封装在 JSON 内部的“自定义业务状态码”来传递。

1. 定义全局状态码枚举类 (ResultCode):

在工程下新建 `common` 包，创建枚举类 `ResultCode`。

Java

```
package com.stu.helloserver.common;

public enum ResultCode {
    // 基础状态码
    SUCCESS(200, "操作成功"),
    ERROR(500, "系统繁忙，请稍后再试"),
    // 权限相关
    TOKEN_INVALID(401, "登录凭证已缺失或过期，请重新登录"); // 模拟未授权状态码

    private final Integer code;
    private final String msg;

    ResultCode(Integer code, String msg) {
        this.code = code;
        this.msg = msg;
    }

    public Integer getCode() { return code; }
    public String getMsg() { return msg; }
}
```

2. 构建泛型载荷 (Result):

在 `common` 包下新建 `Result<T>` 泛型类，用于统一返回给前端的数据外壳。

Java

```
package com.stu.helloserver.common;
```

```

public class Result<T> {
    private Integer code;
    private String msg;
    private T data;

    // 静态工厂方法：成功回调
    public static <T> Result<T> success(T data) {
        Result<T> result = new Result<>();
        result.code = ResultCode.SUCCESS.getCode();
        result.msg = ResultCode.SUCCESS.getMsg();
        result.data = data;
        return result;
    }

    // 静态工厂方法：失败回调
    public static <T> Result<T> error(ResultCode resultCode) {
        Result<T> result = new Result<>();
        result.code = resultCode.getCode();
        result.msg = resultCode.getMsg();
        result.data = null;
        return result;
    }

    // 请自行生成 Getter 和 Setter 方法
}

```

3. 重构 UserController 统一响应:

将上一次实验中的 `UserController` 返回值全部修改为 `Result<T>` 格式。

Java

```

@GetMapping("/{id}")
public Result<String> getUser(@PathVariable("id") Long id) {
    // 将原本直接返回的 String 包装入 Result.success() 中
    String data = "查询成功，正在返回 ID 为 " + id + " 的用户信息";
    return Result.success(data);
}

```

任务 3.2: 编写拦截器 (Interceptor) 与 Config 层挂载

为了保护 Controller 接口不被未授权的用户访问，我们需要在请求到达 Controller 之前，设置一道拦截。

1. 编写自定义鉴权拦截器:

新建 `interceptor` 包，创建 `AuthInterceptor` 类，实现 `HandlerInterceptor` 接口并重写 `preHandle` 方法。

Java

```
import org.springframework.web.servlet.HandlerInterceptor;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class AuthInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response) {
        // 尝试从 HTTP 请求头中截获名为 "Authorization" 的隐藏令牌信息
        String token = request.getHeader("Authorization");

        // 如果没有携带 Token，直接拦截，不放行到 Controller
        if (token == null || token.isEmpty()) {
            response.setContentType("application/json;charset=UTF-8");
            // 构造 401 报错的 JSON 字符串返回给前端
            String errorJson = "{\"code\": 401, \"msg\": \"登录凭证已缺失，请重新\"";
            response.getWriter().write(errorJson);
            return false; // 返回 false 表示拦截打回
        }

        return true; // 令牌存在，返回 true 予以放行
    }
}
```

2. 在 Config 层统筹挂载拦截器:

新建 `config` 包，创建 `WebConfig` 类。注意必须打上 `@Configuration` 注解，使其接

管底层的 MVC 配置。

Java

```
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration // 核心机房配置注解，极度重要
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new AuthInterceptor())
            .addPathPatterns("/api/**") // 拦截 /api 下的所有请求路径
            .excludePathPatterns(
                "/api/users/login", // 保留原本放行的登录接口
                "/api/users",      // 放行：新增用户接口（匹配精确的 /a
                "/api/users/*"      // 放行：获取用户信息接口（匹配 1001
            );
    }
}
```

附加题 1: 实现基于 HTTP 动词的精细化拦截

在 Spring Boot 中，WebConfig 里的 `excludePathPatterns("/api/users/*")` 存在一个致命缺陷：它只能按“URL 路径”放行，完全无视 HTTP 动词。也会把极其危险的删除操作 `DELETE /api/users/1001`。

本任务要求废弃 Config 层针对该路径的宽泛放行，转而在拦截器内部利用 `request`

对象进行极具精准的手动判断。

1. 第一步：收紧 WebConfig 的放行策略

回到 `WebConfig` 类，将刚刚配置的宽泛路径删除。让拦截器重新接管 `/api/users` 的所有操作（只保留登录放行）。

Java

```
@Override
public void addInterceptors(InterceptorRegistry registry) {
    registry.addInterceptor(new AuthInterceptor())
        .addPathPatterns("/api/**")
        .excludePathPatterns("/api/users/login"); // 只在这里放行登录，其他全
}
```

2. 第二步：在 AuthInterceptor 中更改路由

打开的 `AuthInterceptor`，在最上方拦截 Token 之前，加入利用 `getMethod()` 和 `getRequestURI()` 的精准放行逻辑。

Java

```
@Override
public boolean preHandle(HttpServletRequest request, HttpServletResponse response) {

    // 1. 获取本次请求的 HTTP 动词和具体路径
    String method = request.getMethod();
    String uri = request.getRequestURI();

    // 2. 手写细粒度放行规则
    // 规则 A：如果是 POST 请求，且路径精确等于 "/api/users"，则放行（允许注册）
    boolean isCreateUser = "POST".equalsIgnoreCase(method) && "/api/users".equals(uri);

    // 规则 B：如果是 GET 请求，且路径以 "/api/users/" 开头（简易版判断，允许查看信息）
    boolean isGetUser = "GET".equalsIgnoreCase(method) && uri.startsWith("/api/users/");

    // 只要满足上述任一合法公开规则，直接放行，无需查验 Token
    if (isCreateUser || isGetUser) {
        return true;
    }
}
```



```
        return true;
    }

    // 3. 执行严格的 Token 校验（针对 DELETE、PUT 等敏感操作）
    String token = request.getHeader("Authorization");
    if (token == null || token.isEmpty()) {
        response.setContentType("application/json;charset=UTF-8");
        String errorJson = "{\"code\": 401, \"msg\": \"非法操作：敏感动作 [" + m
        response.getWriter().write(errorJson);
        return false;
    }

    return true;
}
```

3. 第三步：使用 Postman 进行防守验证

提交要求

1. **测试结果截图:** 请在 Postman 中分别截取“未携带 Header 被拦截”与“成功携带 Header 并获取统一 JSON 数据”的两张运行结果图。
2. **文档提交:** 完成任务1、2后，将更新后的 `UserController`、`AuthInterceptor` 及 `WebConfig` 核心代码推送至个人仓库分支，并将 **任务成功截图** 粘贴至作业word文档中，命名为《学号_姓名》.docx/.doc，如《xxx_张三.doc》并在提交至慧学平台。